

Technical Guide (English)

Project: Download Bank of Taiwan Exchange Rate

Audience: Developers / DevOps / NetSuite engineers who need to change logic

Chinese version: TECHNICAL.zh-TW.md (./TECHNICAL.zh-TW.md)

User guide: USER_GUIDE.en.md (./USER_GUIDE.en.md)

1. Goals and design principles

Principle	Why
NetSuite must not call BOT directly	BOT uses bot protection; N/https often fails
Stable HTTPS via CDN	Public GitHub repo + jsDelivr
JSON first	NetSuite can JSON.parse without CSV parsing
Editable modules	Download, transform, schedule, and NetSuite write-back are separated

Data flow:

```
rate.bot.com.tw CSV
  |
  ▼
scripts/download_bot_rates.py
- download with curl_cffi (browser TLS impersonation)
- CSV → JSON transform
- write data/
  |
  ▼
GitHub Actions (.github/workflows/update-rates.yml)
- weekday schedule / manual dispatch
- commit + push only when changed
- purge jsDelivr
  |
  ▼
cdn.jsdelivr.net/.../bot-xrt-latest.json
  |
  ▼
netsuite/DownloadBotRates_SS2.js
- N/https.get
- JSON.parse
- save File Cabinet (extendable to Currency Rate)
```

2. Repository layout

```
DownloadBankOfTaiwanExchangeRate/
├── .github/workflows/update-rates.yml # schedule + publish
├── data/
│   ├── bot-xrt-latest.csv           # latest raw CSV
│   ├── bot-xrt-latest.json          # latest JSON (CDN primary)
│   └── bot-xrt-latest.meta.txt       # checksum / fetch time
```

```
| └─ history/          # dated archives (last 90 days only)
|   └─ bot-xrt-YYYY-MM-DD.*
├─ docs/              # documentation
├─ netsuite/DownloadBotRates_SS2.js  # NetSuite Scheduled Script
├─ scripts/download_bot_rates.py     # download + transform core
├─ requirements.txt
└─ README.md
```

3. Local development

3.1 Environment

- Python 3.10+ (CI uses 3.12)
- Dependency: curl_cffi (requirements.txt)

```
python3 -m pip install -r requirements.txt
python3 scripts/download_bot_rates.py
```

3.2 CLI flags

Flag	Purpose
-o / --output	Path for bot-xrt-latest.csv; JSON is written beside it
--no-archive	Skip dated archives under data/history/
--retention-days	Keep only the latest N days in history (default 90)

3.3 Example success output

```
csv=.../data/bot-xrt-latest.csv
json=.../data/bot-xrt-latest.json
csv_bytes=3715
json_bytes=22669
rate_count=19
changed=true
archive_json=.../data/history/bot-xrt-2026-08-11.json
retention_days=90
history_migrated=0
history_pruned=0
```

History policy:

- Directory: data/history/
- Filenames: bot-xrt-YYYY-MM-DD.csv / .json
- Default retention: 90 days (older files are deleted automatically)
- Legacy dated files left under data/ are moved into history/ on each run

changed=false means content matched the previous files (meta still refreshes).

4. Core script: scripts/download_bot_rates.py

4.1 Functions (edit points)

Function	Responsibility	Change when...
download_csv()	Fetch BOT CSV	URL, headers, timeout, impersonate
_to_rate()	Parse numbers; map 0 → null	Keep zeros, precision, filters
_forward_block()	Build forward tenors	Add/remove days
csv_to_payload()	CSV → JSON	Primary place to change JSON schema
write_outputs()	Write CSV/JSON/meta/history	Filenames, archive policy, change detection
migrate_legacy_archives()	Move old dated files into history/	Compatibility with previous layout
prune_history()	Delete files older than retention	Retention policy changes
main()	CLI	Extra args / post-steps

4.2 Key constants

```
BOT_CSV_URL = "https://rate.bot.com.tw/xrt/flcsv/0/day"
FORWARD_DAYS = (10, 30, 60, 90, 120, 150, 180)

BUY_CASH = 2
BUY_SPOT = 3
BUY_FWD_START = 4
SELL_CASH = 12
SELL_SPOT = 13
SELL_FWD_START = 14
```

4.3 BOT CSV column map (0-based)

Each data row is approximately:

Index	Content
0	Currency code (USD)
1	Label “本行買入” (bank buy)
2	Cash buy
3	Spot buy
4–10	Forward buy 10/30/60/90/120/150/180
11	Label “本行賣出” (bank sell)
12	Cash sell
13	Spot sell
14–20	Forward sell 10/30/60/90/120/150/180

If BOT reshuffles columns, update constants + csv_to_payload() only. NetSuite can stay unchanged if the JSON contract remains stable.

4.4 Why curl_cffi?

Plain requests / urllib TLS fingerprints are often challenged and return HTML.

curl_cffi with impersonate="chrome" mimics a browser handshake.

If blocked again, try in order:

1. Switch impersonate (chrome / chrome131 / safari)
2. Adjust Referer / Accept-Language
3. Only then consider Playwright (heavier CI cost)

Keep these failure detectors:

- Content-Type contains text/html
- Body starts with <!DOCTYPE
- No comma in the first 200 bytes (not CSV-like)

4.5 JSON schema (contract)

```
{
  "source": "Bank of Taiwan",
  "sourceUrl": "https://rate.bot.com.tw/xrt/flcsv/0/day",
  "base": "TWD",
  "fetchAtUtc": "ISO-8601",
  "rateCount": 19,
  "rates": [ { "...": "CurrencyEntry" } ],
  "byCurrency": {
    "USD": { "...": "CurrencyEntry" }
  }
}
```

CurrencyEntry:

```
{
  "currency": "USD",
  "cash": { "buy": 31.87, "sell": 32.54 },
  "spot": { "buy": 32.195, "sell": 32.345 },
  "forward": {
    "buy": { "10": 32.214, "30": 32.164, "60": null, "90": null, "120": null, "150": null, "180": null },
    "sell": { "10": 32.318, "30": 32.272, "60": null, "90": null, "120": null, "150": null, "180": null }
  }
}
```

Contract rules:

- NetSuite expects both byCurrency and rates
- Missing quotes are null, not 0 (unless you update NetSuite checks together)
- Prefer additive fields (mid, localized names) for backward compatibility

4.6 Change detection

write_outputs() compares byte equality of:

- bot-xrt-latest.csv
- bot-xrt-latest.json

Either difference ⇒ changed=true.

Note: fetchedAtUtc changes every run, so JSON often changes even when rates do not.
To publish only when rates change, compare a payload without fetchedAtUtc (or compare canonical byCurrency).

Conceptual approach:

```
comparable = {k: v for k, v in payload.items() if k != "fetchedAtUtc"}
new_bytes = json.dumps(comparable, ensure_ascii=False, sort_keys=True).encode()
# compare against previous comparable; still write a fresh fetchedAtUtc when publishing
```

5. GitHub Actions: .github/workflows/update-rates.yml

5.1 Triggers

```
on:
  workflow_dispatch:      # primary: external cron / manual
  repository_dispatch:
    types: [update-bot-rates]
  schedule:               # backup only (UTC)
    - cron: "30 2,7,10 * * 1-5" # Taiwan 10:30 / 15:30 / 18:30
```

Use an external cron as the primary trigger (GitHub's native schedule often skips).

Full setup: SCHEDULING.en.md (./SCHEDULING.en.md).

Local trigger:

```
./scripts/trigger_update.sh
```

5.2 Changing the schedule

Need	Change
Change primary times	Edit cron-job.org (or your crontab)
Change GitHub backup	Edit schedule.cron (UTC)
Manual only	Disable external cron; optionally remove schedule

5.3 Job steps and edit points

1. checkout
2. setup-python + pip install
3. run download_bot_rates.py
4. git diff on data/bot-xrt-latest.csv|json
5. commit / push only if changed
6. purge jsDelivr

Permissions:

```
permissions:
  contents: write
```

Branch protection or custom deploy keys may require extra repo settings.

5.4 Manual CDN purge

```
curl -fsS "https://purge.jsdelivr.net/gh/ChesleyLo/DownloadBankOfTaiwanExchangeRate@main/data/bot-xrt-latest.json"
```

5.5 Repository must be public

jsDelivr cannot read private repos:

```
gh repo edit ChesleyLo/DownloadBankOfTaiwanExchangeRate \  
--visibility public \  
--accept-visibility-change-consequences
```

6. NetSuite: netsuite/DownloadBotRates_SS2.js

6.1 Runtime

- Type: Scheduled Script 2.1
- Entry point: execute(context)
- Modules: N/https, N/file, N/log, N/runtime

6.2 Constants (commonly edited)

Constant	Meaning
CDN_JSON_URL	Default CDN endpoint
FOLDER_ID	File Cabinet folder internal ID
FILE_NAME	Latest filename
DEFAULT_RATE_FIELD	e.g. spot.sell

6.3 Functions (edit points)

Function	Responsibility	Change when...
getConfiguredUrl()	Script param or default URL	Multi-environment URLs
getRateFieldPath()	cash/spot selection	Accounting policy change
parsePayload()	Validate JSON contract	Schema changes
readRate()	Resolve dotted paths	Support forward.sell.30 etc.
saveJson()	Write File Cabinet	Naming / overwrite policy
maybeApplyRates()	Rate write-back extension point	Currency Rate / custom records
execute()	Main flow	Alerts, retries, fallbacks

6.4 Recommended script parameters

ID	Type	Purpose
----	------	---------

custscript_bot_cdn_json_url	Text	CDN URL
custscript_bot_rate_field	Text	spot.sell / cash.buy...
custscript_bot_apply_rates	Checkbox	Enable write-back

6.5 Enabling real Currency Rate writes

maybeApplyRates() currently only log.debugs. To persist rates:

1. Add "N/record" (and N/search / N/format if needed)
2. Map fields for your account's Currency Rate record
3. Handle multi-currency base, effective date, and inverse rates (TWD→USD vs USD→TWD)

Conceptual sketch (must be adapted to your account; do not copy blindly):

```
define(["N/https", "N/file", "N/log", "N/runtime", "N/record"], (
  https, file, log, runtime, record
) => {
  // ...
  function upsertCurrencyRate(fromCode, toCode, exchangeRate, effectiveDate) {
    const rec = record.create({ type: record.Type.CURRENCY_RATE, isDynamic: true });
    // rec.setValue({ fieldId: "...", value: ... }); // account-specific
    return rec.save();
  }
});
```

readRate() already supports dotted paths such as forward.sell.30.

6.6 Restrict currencies

Inside maybeApplyRates():

```
const ALLOWED = ["USD", "EUR", "JPY", "CNY"];
currencies.filter((c) => ALLOWED.indexOf(c) !== -1).forEach(/* ... */);
```

6.7 Error-handling ideas

Already present:

- Non-2xx HTTP → throw
- Invalid JSON → throw
- Missing byCurrency/rates → throw

Optional enhancements:

- Retry 2–3 times for transient CDN failures
- Email / custom record on failure
- Fallback to GitHub Raw URL

7. Cookbook

7.1 Add mid = (buy + sell) / 2

After building entry in csv_to_payload():

```
def _mid(side: dict) -> float | None:
    b, s = side.get("buy"), side.get("sell")
    if b is None or s is None:
        return None
    return round((b + s) / 2, 6)

entry["spot"]["mid"] = _mid(entry["spot"])
entry["cash"]["mid"] = _mid(entry["cash"])
```

NetSuite can then read spot.mid.

7.2 Historical date CSV

BOT may support:

```
https://rate.bot.com.tw/xrt/flcsv/0/YYYY-MM-DD
```

Parameterize:

```
parser.add_argument("--date", help="YYYY-MM-DD; default=day")
# url = f"https://rate.bot.com.tw/xrt/flcsv/0/{args.date or 'day'}"
```

7.3 Publish to another CDN / object store

Replace the final Actions step with S3/R2/Azure upload and update the NetSuite URL.
Keep the JSON contract stable so SuiteScript changes stay minimal.

7.4 Rename published files

Update all of:

1. write_outputs() filenames
2. Actions purge paths
3. NetSuite CDN_JSON_URL / FILE_NAME
4. URLs in user docs

8. Test checklist

Download / transform

- [] python3 scripts/download_bot_rates.py succeeds
- [] JSON loads with json.load
- [] byCurrency.USD.spot.sell is a number
- [] Missing quotes are null
- [] Blocked/offline run exits non-zero

Actions

- [] Manual workflow run succeeds
- [] Commit updates data/bot-xrt-latest.json
- [] CDN URL returns HTTP 200
- [] After purge, CDN serves the new content

NetSuite

- [] Execute Now succeeds
 - [] File Cabinet has latest + dated files
 - [] Execution Log shows USD spot sell
 - [] (If enabled) Currency Rate / custom records are correct
-

9. Troubleshooting

Symptom	Likely cause	Action
HTML Challenge downloaded	WAF	Verify curl_cffi, switch impersonate
CDN 404 file not found	Private repo / wrong path	Make public; verify path
Stale CDN content	Cache	Purge; temporarily use Raw URL
Actions cannot push	Token/permissions	contents: write, branch rules
NetSuite connectivity failure	Allowlist	Permit cdn.jsdelivr.net
JSON parse error	HTML/error page returned	Check URL and HTTP status

10. Security and compliance

- Never commit secrets to a public repo
 - Cite Bank of Taiwan as the source
 - CDN is a mirror and may lag; define reconciliation for finance-critical use
 - Rate write-back is sensitive — require code review
-

11. Maintenance tips

1. Evolve the JSON contract additively (add fields before removing)
 2. Update bilingual docs + README for material changes
 3. If BOT changes CSV layout, fix Python first and confirm NetSuite still works
 4. Quarterly spot-check CDN values against the official BOT page
-

12. Document index

Document	Description
TECHNICAL.zh-TW.md (./TECHNICAL.zh-TW.md)	Technical guide (Chinese)
TECHNICAL.en.md (./TECHNICAL.en.md)	This technical guide (English)
USER_GUIDE.zh-TW.md (./USER_GUIDE.zh-TW.md)	User guide (Chinese)
USER_GUIDE.en.md (./USER_GUIDE.en.md)	User guide (English)